Language Modeling and Sequence Learning

Author: Lam Nguyen

Available: Jan 15, 2026 12:00am until Feb 25, 2026 11:59pm

Overview

This assignment examines probabilistic and neural approaches to language modeling, sequence modeling with recurrent neural networks, and sequence-to-sequence architectures. All questions are based strictly on the material covered in **Lectures 8, 9, and 10**. Each question consists of two parts and requires precise, technically grounded answers.

Note: This is an individual assignment. Clarity, correctness, and depth of reasoning are expected.

Submission Guidelines

- Submit a single PDF document
- Clearly label each question and part (A, B)
- Answers should be concise, technical, and well-structured
- Do not include material beyond Lectures 8–10
- Each question is worth 5 pts (total: 40 pts)

Questions

Question 1: Language Modeling Fundamentals (Lecture 8)

(A) Define a language model formally and explain how language modeling is framed as a next-word prediction problem.

Answer: A language model is a system that assigns probabilities to word sequences and predicts the next word based on prior context. For a sequence of words, the model computes a probability distribution over the next word where $x^{(t+1)}$ can be any word in the vocabulary V. Language modeling is framed as next-word prediction because the model tries to predict which word is most likely to follow given the context.

(B) Explain how assigning probabilities to entire sentences follows from next-word prediction.

Answer: A language model assigns probability to a sequence by factorizing it into a product of conditional probabilities using the chain rule. Each term in this product is a next-word prediction conditioned on all preceding words. By computing these conditional probabilities sequentially and multiplying them together, the model assigns a probability to the complete sentence based on how likely the word order will occur during use.

Question 2: N-Gram Language Models (Lecture 8)

(A) Explain the n-gram independence assumption and how it simplifies probability estimation.

Answer: The n-gram independence assumption states that the next word depends only on the previous n−1 words, not on the entire preceding sequence. This simplifies probability estimation. Instead of estimating probabilities over exponentially many possible histories, the model only needs to count occurrences of n-grams in the training corpus and compute probabilities as simple ratios.

(B) Using an example, explain how increasing the value of n affects modeling power and data requirements.

Answer: Here is an example of predicting the next word after the phrase "students opened their". In a bigram model (n=2), the model only sees "their" and might predict common words like "own" or "first". In a 4-gram model (n=4), the model sees "students opened their" and can make more contextually appropriate predictions like "books" or "exams". Increasing n improves modeling power by capturing longer dependencies and more specific contexts. However, it exponentially increases data requirements. For example, for a vocabulary of 10,000 words, bigrams require storing up to 10^8 entries, while trigrams require 10^{12} entries. Many valid longer n-grams never appear in finite training data, leading to severe sparsity problems.

Question 3: Sparsity and Smoothing (Lecture 8)

(A) Explain why data sparsity is a fundamental problem in n-gram language models.

Answer: Data sparsity is a fundamental problem because as n increases, many valid word sequences never appear in training data. This results in zero probabilities for unseen n-grams. This occurs due to:finite training data that cannot cover all possible word combinations, rare or novel phrases that are grammatically valid but infrequent and finally, exponential growth of possible n-grams with vocabulary size and n value. When a model assigns zero probability to an unseen but valid sequence, it fails at prediction and generation tasks. This makes n-gram models unreliable for real-world applications where novel combinations are common.

(B) Describe smoothing, backoff, or interpolation and explain how these techniques partially address sparsity.

Answer: These techniques redistribute probability mass to handle unseen n-grams: **Smoothing** adds small counts to all n-grams using techniques like Laplace smoothing or Kneser-Ney smoothing for context-sensitive adjustments, ensuring no n-gram has exactly zero probability. **Backoff** uses lower-order n-grams when higher-order n-grams are unavailable—if a 4-gram wasn't seen, the model backs off to a trigram, bigram, or unigram estimate. **Interpolation** combines multiple n-gram orders by taking a weighted average of probabilities from different n values. These techniques somewhat mitigate sparsity by ensuring the model can still make predictions for unseen sequences. However, they cannot fully overcome the fundamental limitation that count-based models don't generalize beyond exact matches.

Question 4: Fixed-Window Neural Language Models (Lecture 8)

(A) Describe how fixed-window neural language models predict the next word using a feedforward network.

Answer: Fixed-window neural language models predict the next word by inputting a fixed number of preceding words through a feedforward NN. The model takes the previous n words, embeds each word into a continuous vector representation, joins these embeddings into a single input vector, and passes it through some hidden layers. The final layer produces scores over the entire vocabulary, which are converted to probabilities using softmax. The model is trained to maximize the probability of the correct next word. Unlike n-grams that rely on discrete counts, neural models use continuous representations that allow generalization based on similarity between word contexts.

(B) Explain why fixed-window neural models still fail to capture long-range dependencies.

Answer: Fixed-window neural models fail to capture long-range dependencies because they only consider a fixed number of preceding words, regardless of sentence length. Information outside the window is discarded. Even if earlier words are crucial for understanding, the model cannot condition on them once they fall outside the fixed window. This means that no matter how the window size is chosen, there will always be dependencies that span beyond it, making the model unable to represent long contextual relationships needed for real world language comprehension.

Question 5: Recurrent Language Models (Lecture 9)

(A) Explain how recurrent neural networks model sequential dependencies using hidden states.

Answer: RNNs model sequential dependencies by maintaining a hidden state $h^{(t)}$ that is updated at each timestep as a function of both the current input $x^{(t)}$ and the previous hidden state $h^{(t-1)}$. This recurrence introduces directed cycles that allow information to persist across time, giving the model memory of past inputs. The hidden state serves as a compressed representation of all prior context in the sequence. At each timestep, the RNN computes $h^{(t)} = f(h^{(t-1)}, x^{(t)})$, where f is typically a nonlinear transformation. This design allows the model to condition predictions on the entire preceding sequence (summarized in the hidden state) rather than just a fixed window, enabling it to capture dependencies over arbitrarily long sequences.

(B) Explain how an RNN language model generates text autoregressively.

Answer: An RNN language model generates text autoregressively by repeatedly predicting the next word and feeding it back as input for the subsequent prediction. The process starts with an initial hidden state and a start token. These are the steps: first the model embeds the current word, next it updates the hidden state based on this embedding and the previous hidden state, next it computes scores over the vocabulary, then applies softmax to get a probability distribution, and finally, samples or selects the most likely next word. The predicted word is then fed back as input for the next timestep, and the loop continues until an end-of-sequence token is generated. This autoregressive generation allows the model to produce sequences of arbitrary length while maintaining coherence through the evolving hidden state.

Question 6: Training RNNs and Backpropagation Through Time (Lecture 9)

(A) Describe backpropagation through time (BPTT) and explain why the RNN must be unrolled during training.

Answer: Backpropagation through time (BPTT) is a specialized training algorithm for RNNs that computes gradients by unrolling the RNN across time. The RNN must be unrolled because the same parameters are reused at every timestep, and gradients must account for all these repeated applications. Unrolling converts the recurrent structure into a deep feedforward computation graph where each timestep becomes a layer. Gradients are then computed in reverse temporal order, summing contributions from each timestep where a parameter appears. This allows the model to learn how parameter updates affect predictions across the entire sequence, properly attributing to parameters based on their total impact throughout time.

(B) Explain why training RNNs is more challenging than training feedforward neural networks.

Answer: Training RNNs is more challenging because:

1. **Temporal dependencies**: Gradients must be propagated backward through many timesteps, creating very deep computation graphs that are numerically unstable.
2. **Vanishing/exploding gradients**: Repeated multiplication of the same weight matrices during backpropagation causes gradients to either vanish (approach zero) or explode (grow exponentially), making it difficult to learn long-range dependencies.
3. **Computational cost**: BPTT requires storing activations for all timesteps and computing gradients across the full sequence, which is memory and time intensive.
4. **Sequential processing**: Unlike feedforward networks where examples are independent, RNNs process sequences sequentially, limiting parallelization opportunities. These factors make RNN optimization require careful tuning of learning rates, gradient clipping, and architectural choices.

Question 7: Vanishing Gradients and Gated Architectures (Lecture 9)

(A) Explain the vanishing gradient problem in recurrent neural networks.

Answer: The vanishing gradient problem occurs when gradients become exponentially smaller as they are backpropagated through time in RNNs. During BPTT, gradients are computed through repeated multiplication of the weight matrices and derivatives of activation functions. When these multiplied values are less than 1, the gradient shrinks exponentially with the number of timesteps. After propagating backward through many steps, gradients start to approach zero, making it nearly impossible for the model to learn dependencies between distant timesteps. This prevents RNNs from capturing long-range dependencies because earlier timesteps receive almost no learning signal, and their contribution to the loss disappears during training.

(B) Explain how gated architectures such as LSTMs or GRUs mitigate this problem.

Answer: Gated architectures like LSTMs and GRUs mitigate the vanishing gradient problem by introducing gating mechanisms that control data flow through the network. These gates (input, forget,

and output gates in LSTMs; reset and update gates in GRUs) learn when to retain, update, or discard data in the hidden state. Crucially, they create additive rather than purely multiplicative paths for gradient flow. Fpr example, LSTM cell states use element-wise addition to incorporate new information. Addition instead of multiplication prevents the vanishing gradient problem.

Question 8: Sequence-to-Sequence Models and Attention (Lecture 10)

(A) Explain the encoder–decoder architecture and how sequence-to-sequence models perform conditional generation.

Answer: The encoder–decoder architecture consists of two neural components. First there is the **encoder** that processes the variable-length input sequence and produces a representation. Then there is the **decoder** that generates a variable-length output sequence conditioned on that representation. The encoder reads the input sequence, updating its hidden state at each timestep. After the encoder finishes processing the final input token, it outputs a context vector summarizing the entire sequence. The decoder uses this context vector to initialize the hidden state and then generates the output sequence autoregressively. This functions as a conditional language model where each prediction depends on previously generated tokens and the encoded input.

(B) Explain why attention mechanisms improve sequence-to-sequence models compared to fixed-length context representations.

Answer: Attention mechanisms improve Seq2Seq models by addressing the data bottleneck created by compressing the entire input sequence into a single fixed-length context vector. In basic encoder-decoder models, all input information must pass through this single vector, which becomes inadequate for long sentences as critical details are lost or diluted. Attention allows the decoder to dynamically focus on different parts of the input sequence at each decoding step by computing attention weights over all encoder hidden states. Attention then forms a weighted combination that emphasizes relevant input tokens. This gives the decoder selective, direct access to the full input representation rather than relying solely on the compressed context vector. Attention significantly improves performance on long sequences, enables better alignment between input and output, and provides interpretability by showing which input positions the model focuses on when generating each output word.